FPT UNIVERSITY

PROJECT REPORT

DESIGN AND VERIFICATION OF AN AM (DSB-FC) MODULATOR ON FPGA USING VERILOG RTL

Team: 3

Class: IC2010

Team members: CAO HỮU AN

PHẠM MINH TÙNG

MAI THẾ TÂM

LÊ THẾ TRƯỜNG

PHAN ANH MINH

TRẦN MINH TRIẾT

Lecturer: Dr. Phạm Thế Vinh

HO CHI MINH CITY, NOVEMBER 2026

TABLE OF CONTENTS

1. INTRODUCTION
2. THEORETICAL & FIXED-POINT CONTRACT
   2.1. AM DSB-FC Expression and Integer Conversion
   2.2. Fixed-Point Contract Details
3. HARDWARE ARCHITECTURE
   3.1. System Block Diagram
   3.2. Module Index and Functionality
   3.3. Top-Level Interface Specification (am_modulator_top)
4. VERIFICATION
   4.1. Self-Checking Testbench (tb_am_top.v)
   4.2. Comparison Verification with Python Golden Model (RTL vs Reference)
   4.3. Python Unit Tests
5. SYNTHESIS RESULTS & ANALYSIS
   5.1. Hypotheses Under Test
   5.2. Vivado Batch Sweep Synthesis Results Frame
6. LIMITATIONS
7. EVALUATION & COMPARISON
   7.1. Comparison between Full-Wave ROM and Quarter-Wave ROM Architectures
   7.2. Comparison between ROM-based NCO Approach and Algorithmic Approximation Approach (CORDIC / Polynomial)
8. TEAM MEMBERS' CONTRIBUTION
9. REFERENCES

1. INTRODUCTION

Amplitude Modulation (AM) with Double Sideband Full Carrier (DSB-FC) is one of the most fundamental techniques in communications signal processing. This project realizes the complete hardware processing flow (RTL Design) for an AM modulator using the Verilog HDL language.

Design Specifications:

- **System Clock Frequency (fclk):** 100MHZ
- **Datapath Width:** 12-bit signed for sine wave samples, 16-bit signed for output am_out.
- **Phase Accumulator:** 32-bit for high frequency resolution.
- **Pipeline Latency:** Exactly 3 clock cycles from input to am_out.
- **ROM Architecture:** Supports 2 lookup methods (Full-wave ROM and Quarter-wave ROM) to compare resource optimization.
- **Target Synthesis Device:** Xilinx Artix-7 (xc7a35tcpg236-1).

2. THEORETICAL & FIXED-POINT CONTRACT

2.1. AM DSB-FC Expression and Integer Conversion

Theoretical expression of the AM DSB-FC signal:

$$s(t) = A_c \cdot [1 + \mu \cdot m(t)] \cdot \cos(2\pi \cdot f_c \cdot t)$$

In fixed-point digital hardware, the integer value 2047 corresponds to a scaling factor of 1.0 (scale factor 211=2048). To avoid costly floating-point calculations, the theoretical formula is decomposed into 3 integer operations executed per clock cycle:

1. **Message Scaling (Multiplication & Bit-shift):**

$$\text{msg_term} = (\text{amp_msg} \times \text{msg}) \gg 11$$

2. **Envelope Offset Generation:**

$$\text{env} = \text{amp_car} + \text{msg_term}$$

3. **Carrier Modulation:**

$$\text{am_out} = (\text{env} \times \text{car}) \gg 11$$

Note on Modulation Index:

$$\mu = \frac{\text{amp_msg}}{\text{amp_car}}$$

To ensure the envelope env >= 0 (avoiding envelope folding), the design adheres to the condition amp_msg <= amp_car. When amp_car = 0, the system automatically switches to Double Sideband Suppressed Carrier (DSB-SC) modulation mode on the exact same hardware logic.

2.2. Fixed-Point Contract

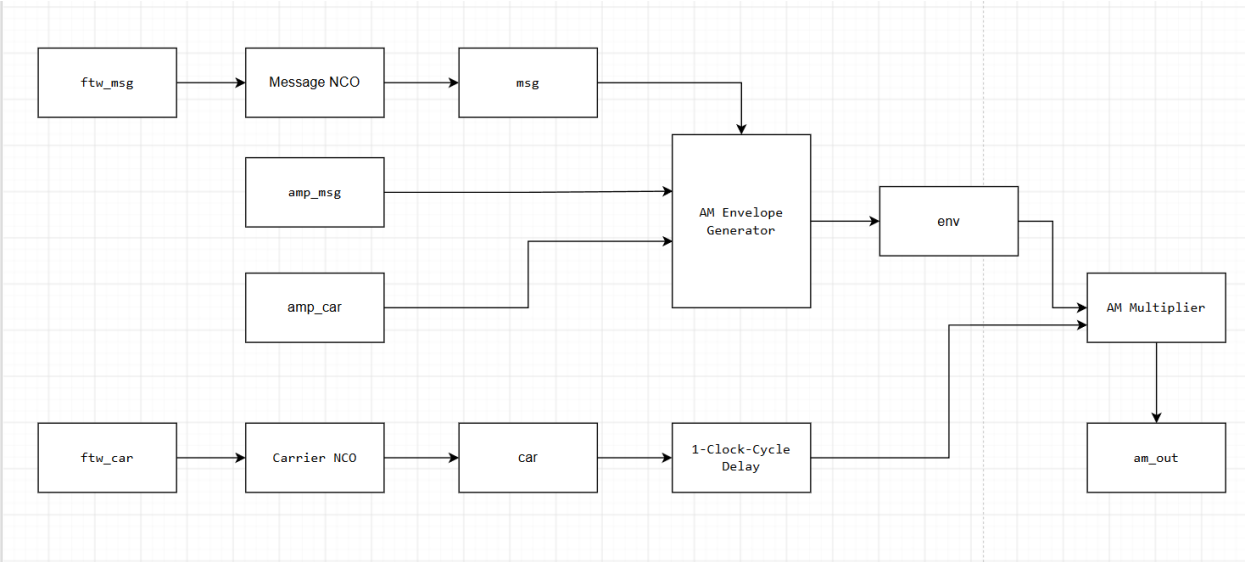
Stage	Expression	Bit-width	Value Range
ROM Sample	car, msg	12-bit signed	-2048 ….+2047
Message Scaling	(amp_msg * msg) >>> 11	25-bit product -> giữ 14-bit	Signed operation
Envelope	amp_car + msg_term	14-bit signed	1000….2999 (with default config)
Output	(env * car) >>> 11	26-bit product -> giữ 16-bit	-2998…+2997((with default config)

Two core rules in RTL design:

1. **Bit-shift operation >>>:** Arithmetic right shift of a signed number rounds towards negative infinity. Python's >> shift operator on signed integers behaves identically.
2. **Signed Casting:** amp_msg is an unsigned number, while msg is a signed number. RTL must sign-extend amp_msg with a leading 0 bit signed({1'b0, amp_msg}) prior to performing multiplication. Without explicit signed type casting, the negative half-cycle of the msg signal will be calculated incorrectly, even if the waveform visually appears acceptable.

3. HARDWARE ARCHITECTURE

3.1. Block Diagram



(Note: The carrier signal car is delayed by 1 clock cycle in am_modulator_top to eliminate timing skew, ensuring both the carrier and envelope enter the am_multiplier module during the same clock cycle).

3.2. Module Index

Module	Function	**Parameters**
phase_acc	Free-running 32-bit phase accumulator with overflow wrap-around.	PW = 32
sine_rom_full	Full-period sine lookup ROM table (2AW elements), 2 register stages	AW, DW, MEM_FILE
sine_rom_quarter	Quarter-period lookup table (2AW-2 elements) combining address folding & sign inversion logic.	AW, DW, MEM_FILE
nco	Integrates phase_acc and ROM; selects ROM architecture via generate construct.	USE_QUARTER_ROM
am_envelope	Signed Unsigned multiplication, bit shifting, envelope offset addition.	DW, EW = DW + 2
am_multiplier	Multiplies envelope with carrier and formats output data.	DW, EW, OW = 16
am_modulator_top	Top-level module, connects all blocks and aligns pipeline delays for carrier.	All parameters above

3.3. am_modulator_top

Port	Direction	Width	Description
Clk	In	1	100 MHz system clock
rst	In	1	Synchronous reset signal, active-high
ftw_car	In	32	Carrier frequency tuning word
ftw_msg	In	32	Message signal frequency tuning word
amp_car	In	12	Carrier amplitude (Unsigned)
amp_msg	In	12	Message signal amplitude (Unsigned)
am_out	Out	16	Modulated AM signal (Signed 16-bit)

Default Stimulus:

- ftw_car = 42_949_673 (corresponding to f_car = 1 MHz)
- ftw_msg = 858_993 (corresponding to f_msg = 20 kHz)
- amp_car = 2000, amp_msg = 1000

4. VERIFICATION

The verification process is implemented across a strict 3-tier structure to guarantee bit-exact accuracy between RTL hardware and theoretical models.

4.1. Self-Checking Testbench (tb_am_top.v)

Self-checking testbench based on theoretical properties obtained directly from simulation waveforms:

- **Reset Signal:** Successfully drives am_out back to 0.
- **Envelope Peak:** Reaches expected target value of approximately 2999
- (amp_car + amp_msg).
- **Envelope Trough:** Reaches expected target value of approximately 1000 (amp_car - amp_msg).
- **Maximum Amplitude of am_out:** Reaches 2997 (Positive) and -2998 (Negative).

Modulation Index Measurement: Recalculated value matches exactly 0.50.

$$\mu = \frac{E_{max} - E_{min}}{E_{max} + E_{min}}$$

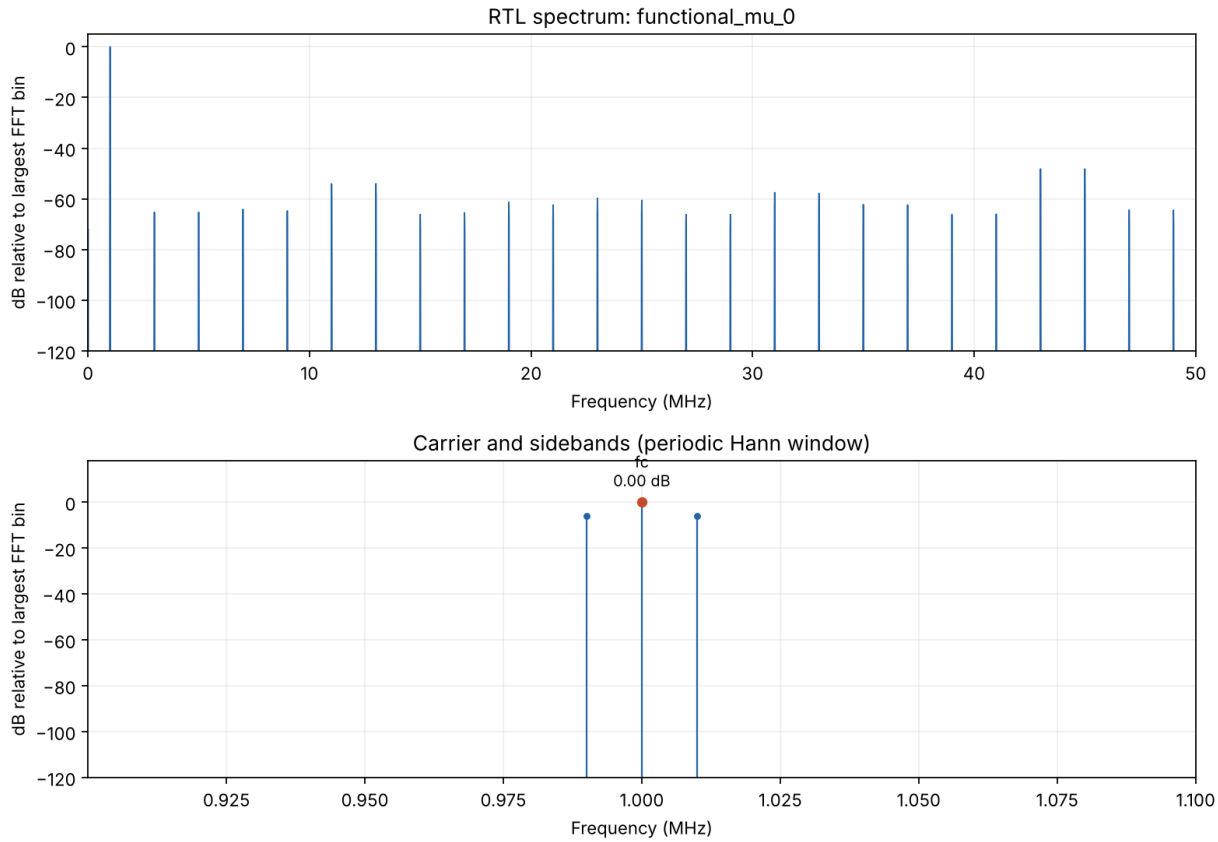
Result: 7/7 Checks PASS.

4.2. Comparison Verification with Python Golden Model (RTL vs Reference)

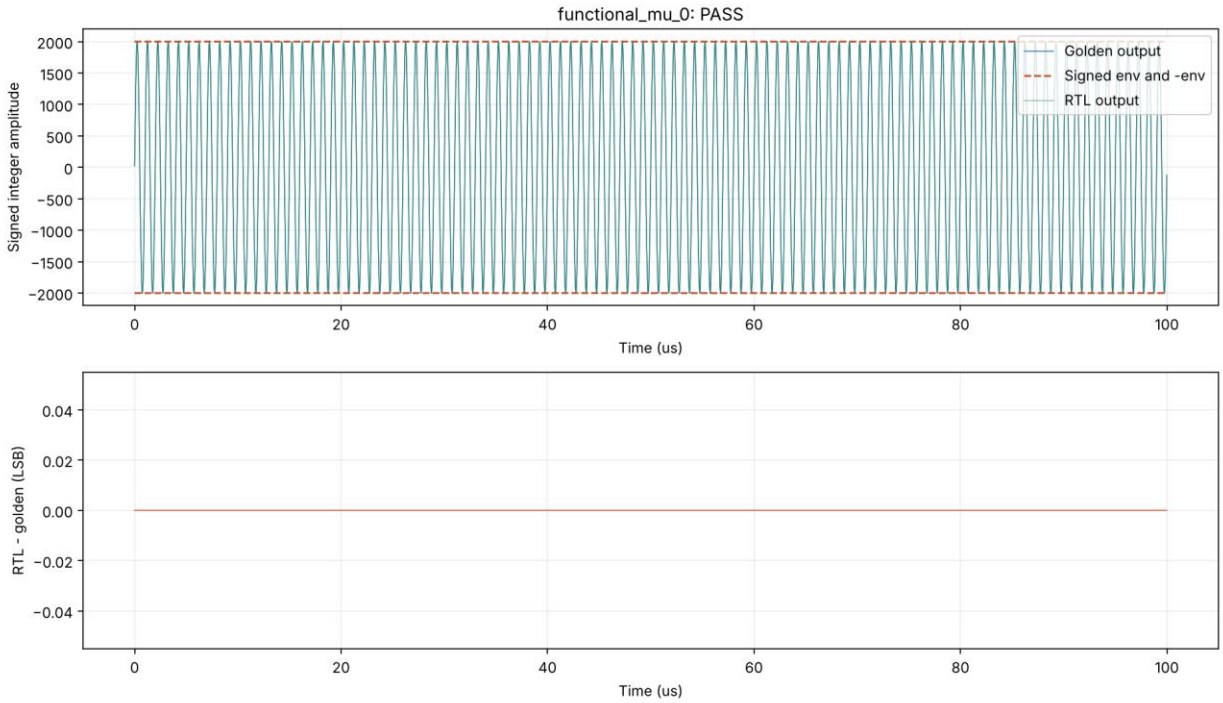
The script run_verification.py combined with tb_am_top_filebased.v was executed across 13 different simulation configurations (10,000 data samples per configuration):

$\mu = 0$ Case (No Modulation):

spectrum

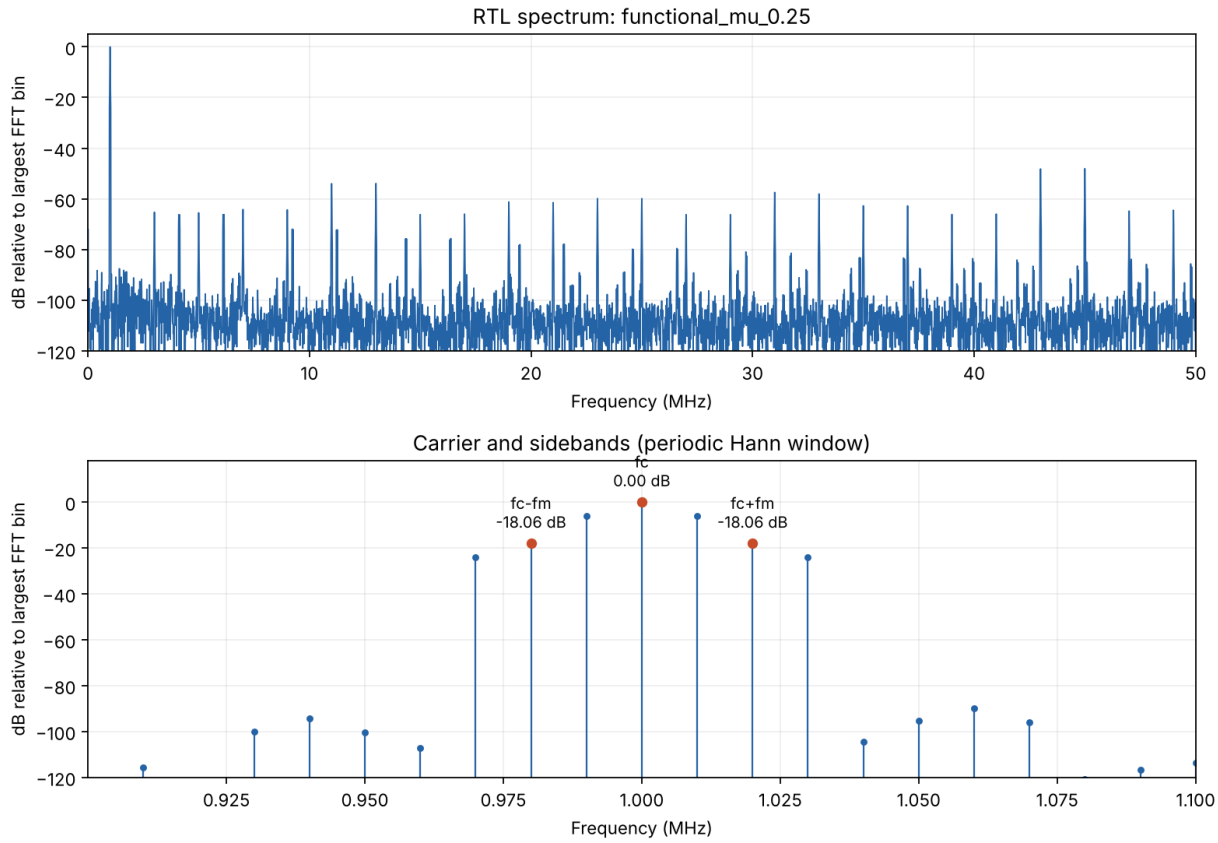


Waveform

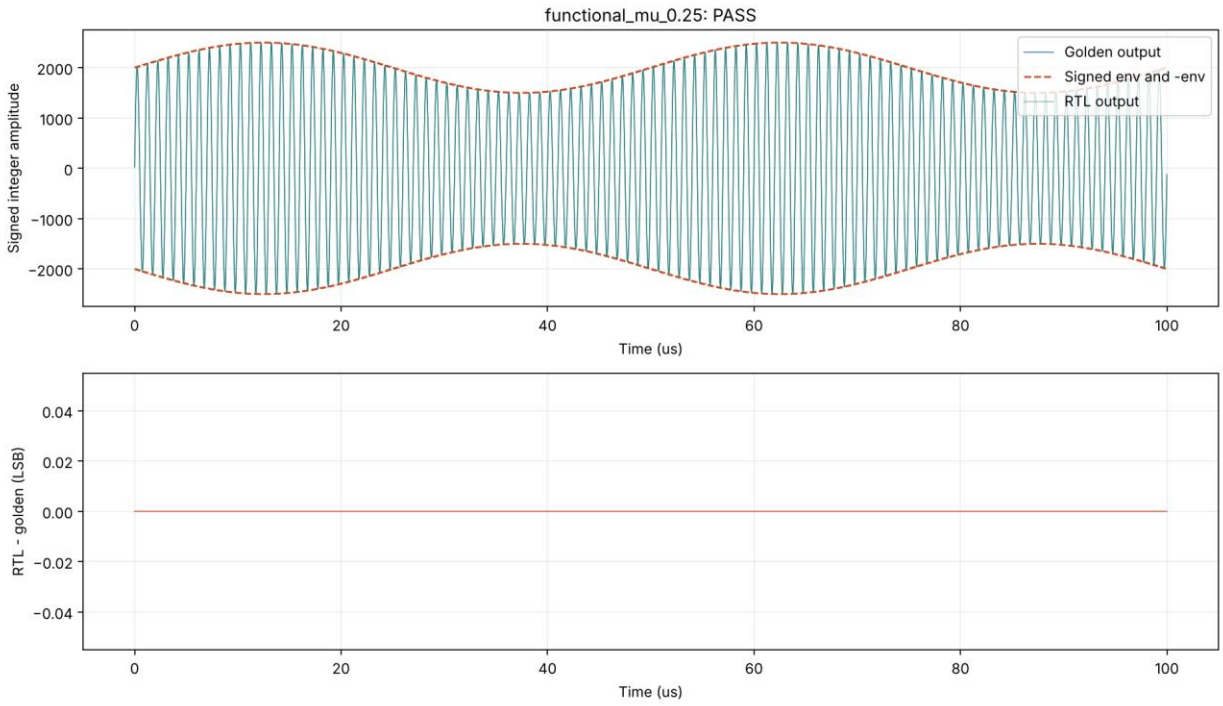


$\mu = 0.25$ Case:

Spectrum:

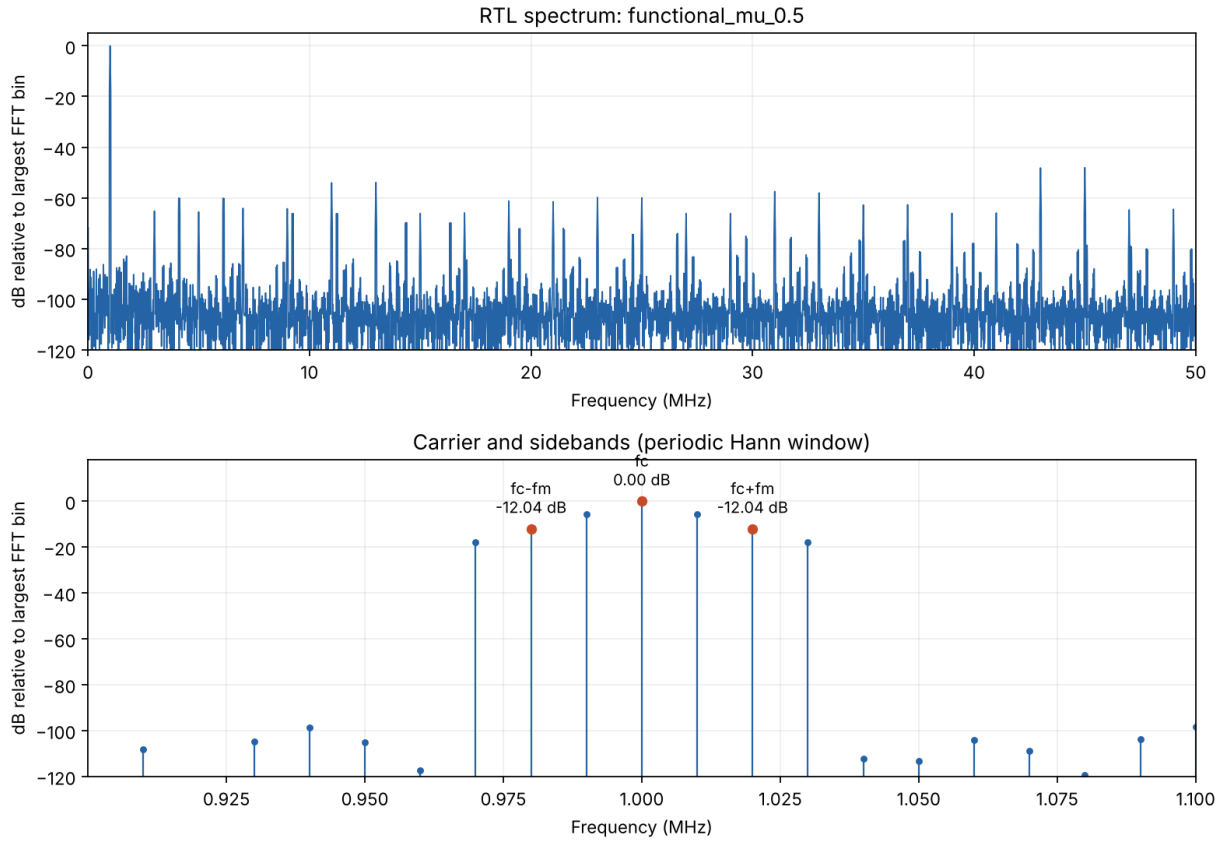


Waveform:

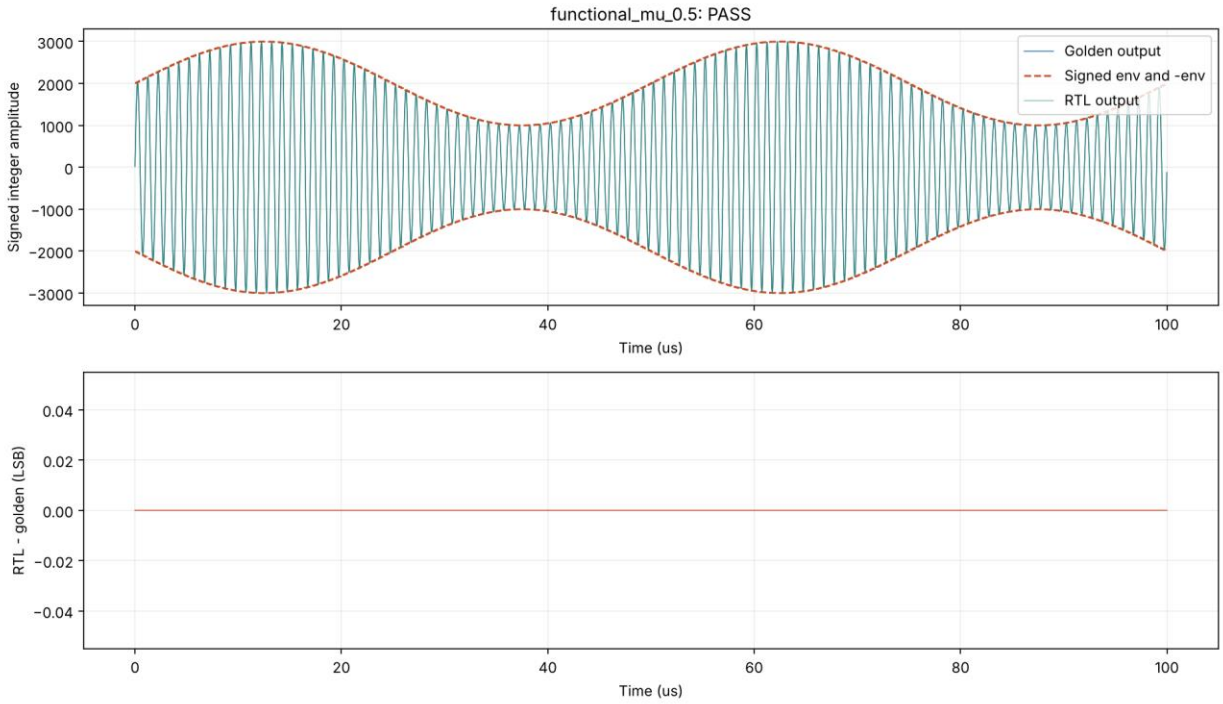


$\mu = 0.50$ Case (Default):

Spectrum:

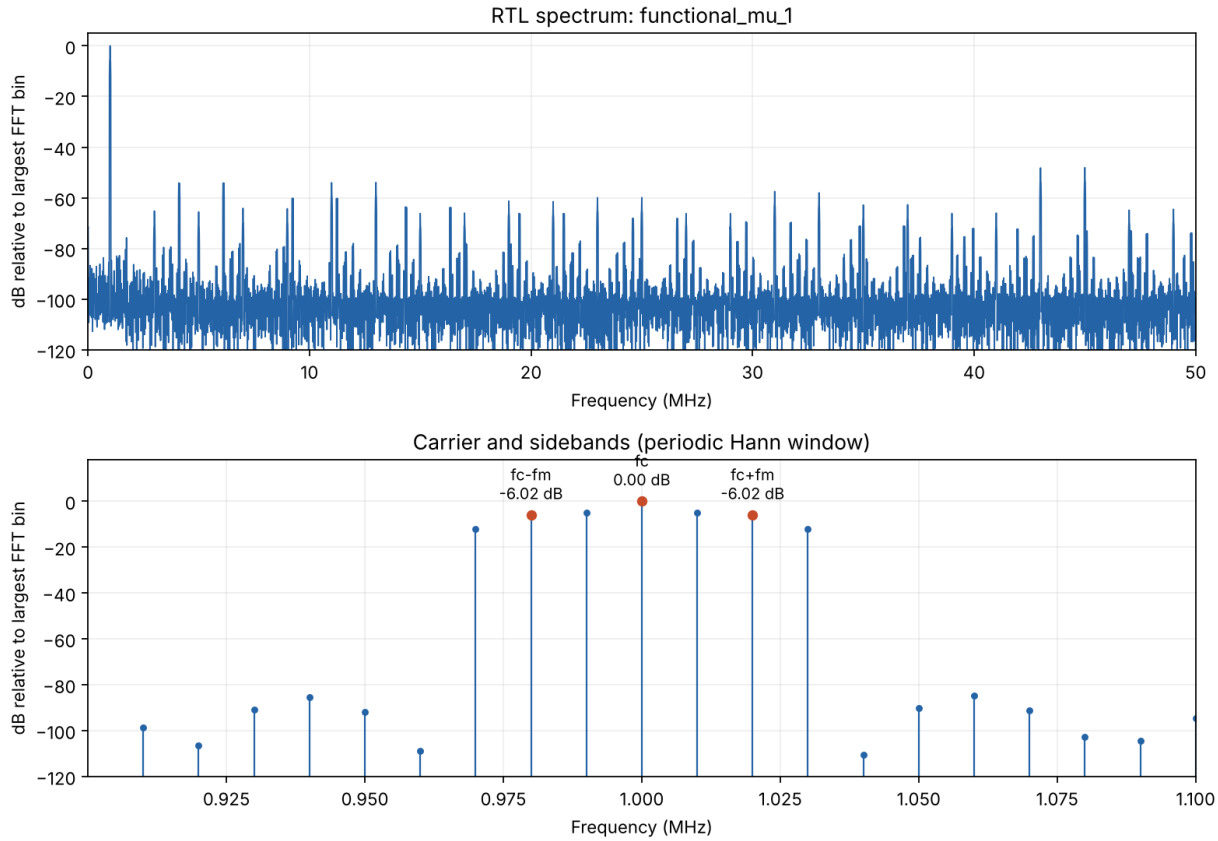

RTL spectrum: functional_mu_0.5
dB relative to largest FFT bin
Frequency (MHz)
Carrier and sidebands (periodic Hann window)
fc-fm
-12.04 dB
fc
0.00 dB
fc+fm
-12.04 dB
dB relative to largest FFT bin
Frequency (MHz)

Waveform:

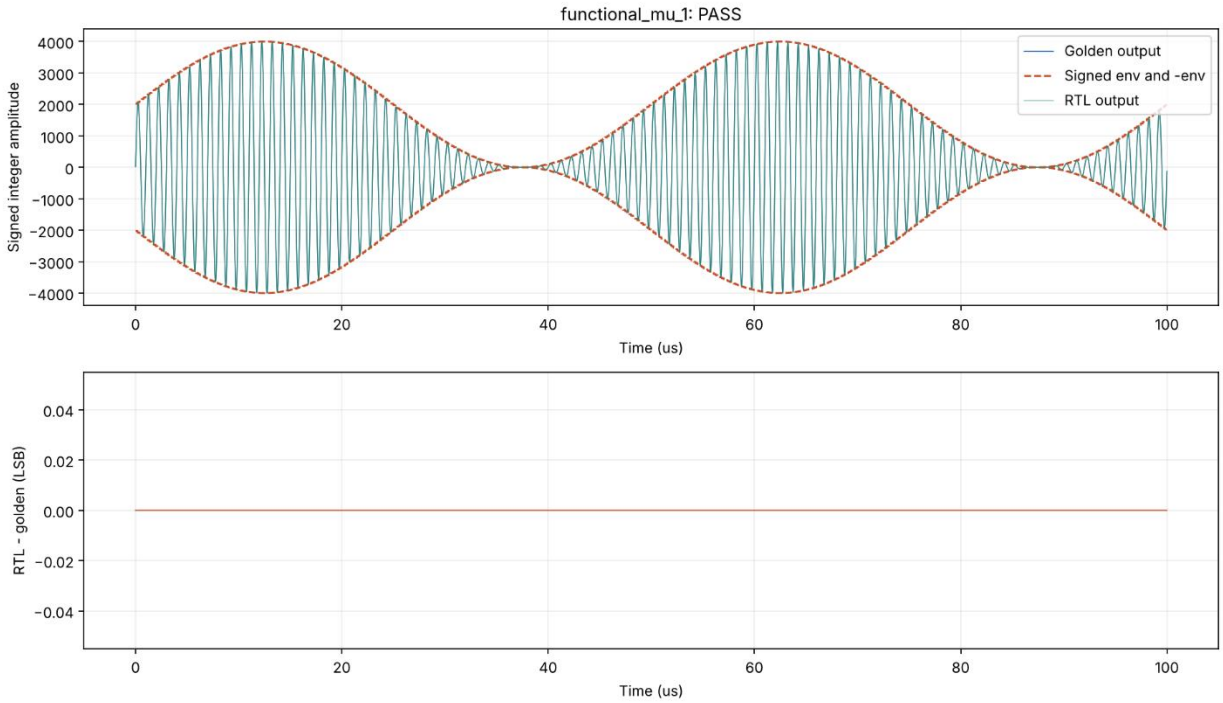

functional_mu_0.5: PASS
Golden output
Signed env and -env
RTL output
Signed integer amplitude
Time (us)
RTL - golden (LSB)
Time (us)

$\mu = 1.00$ Case (100% Modulation):

Spectrum:

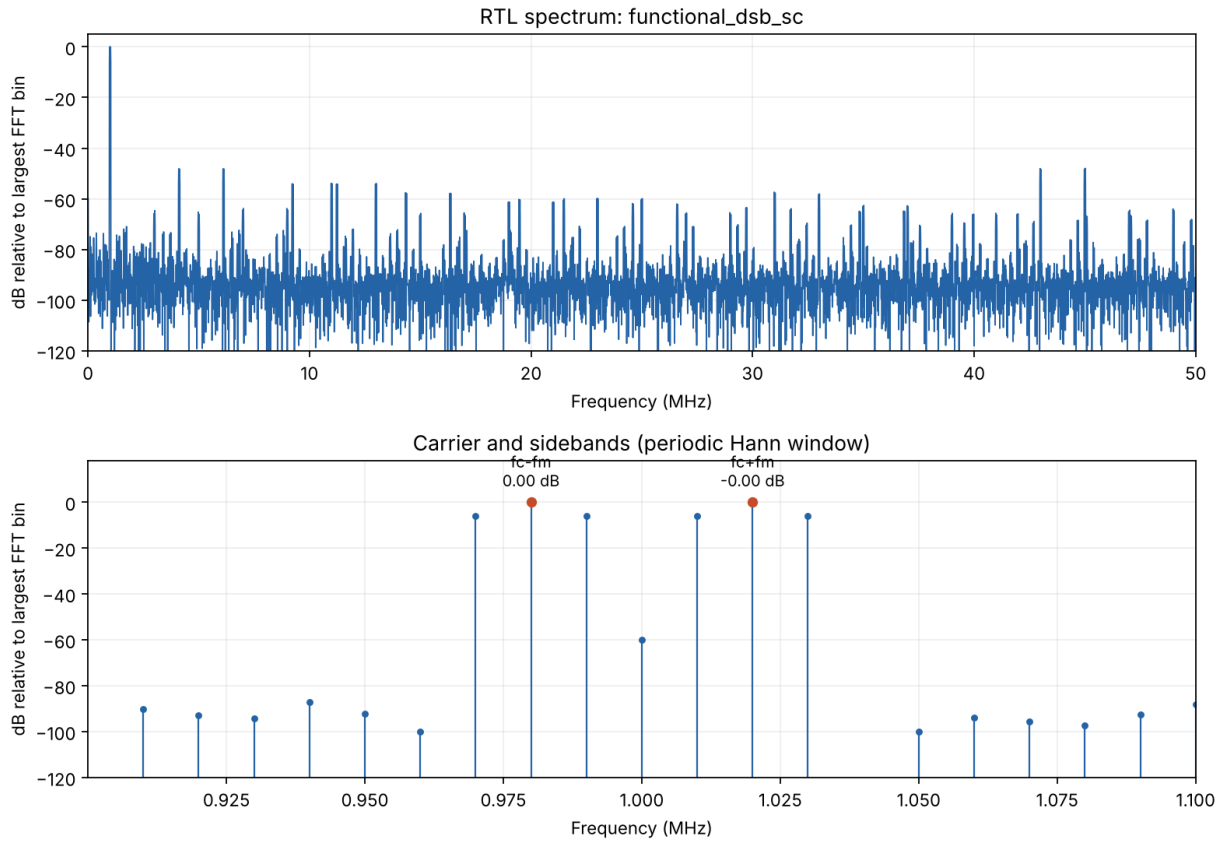


Waveform:

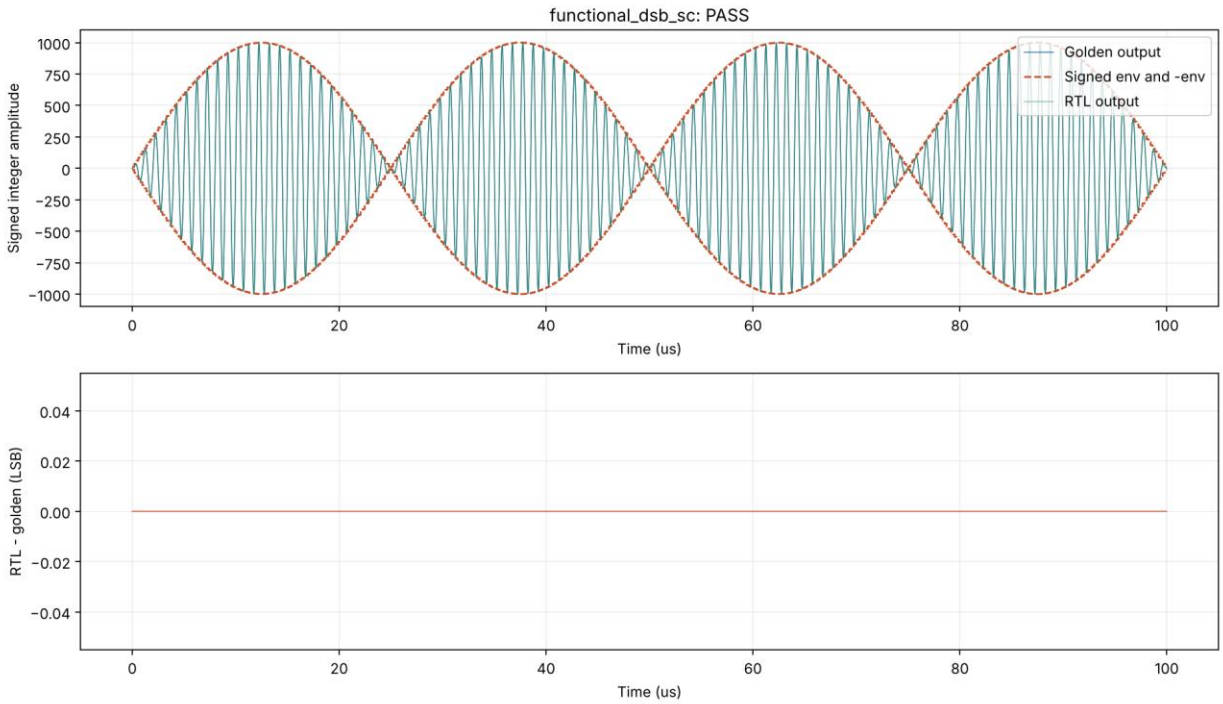


DSB-SC Mode (`amp_car = 0`):

Spectrum:

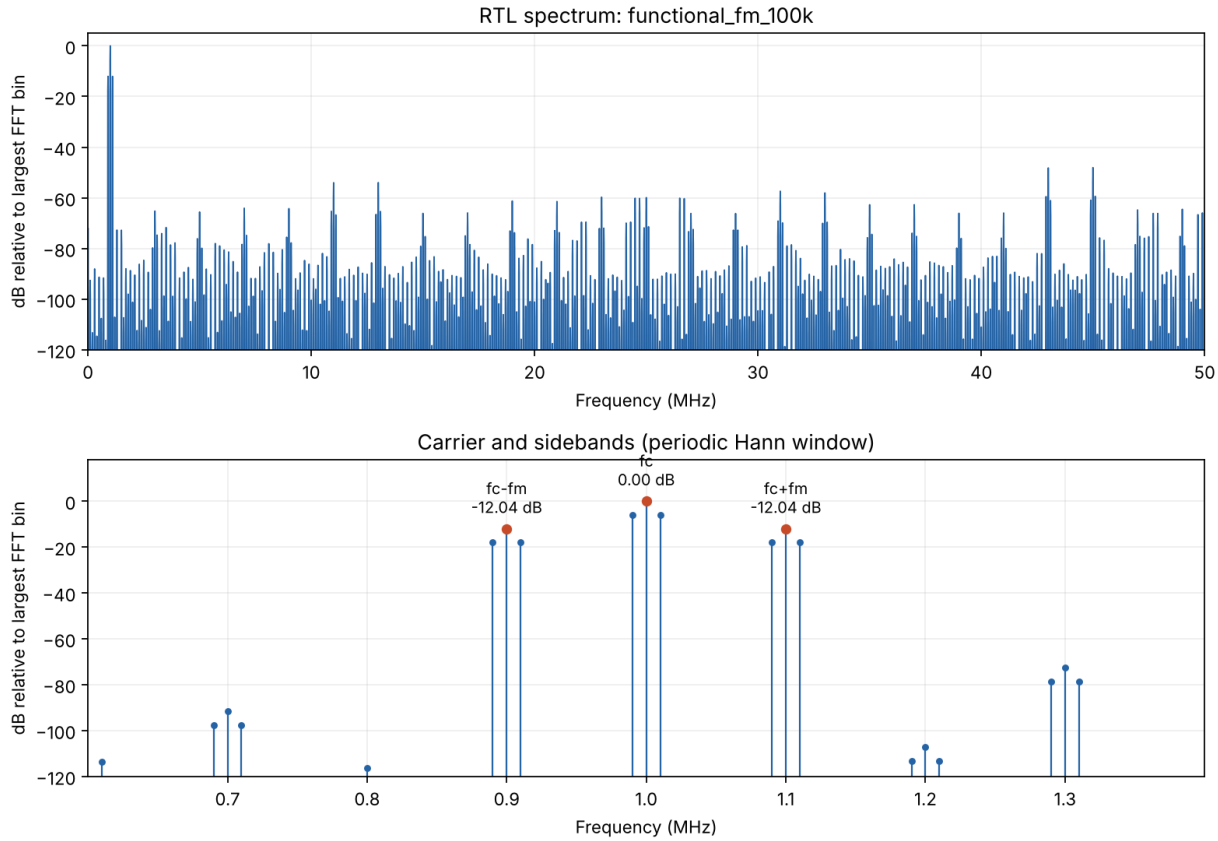


Waveform:

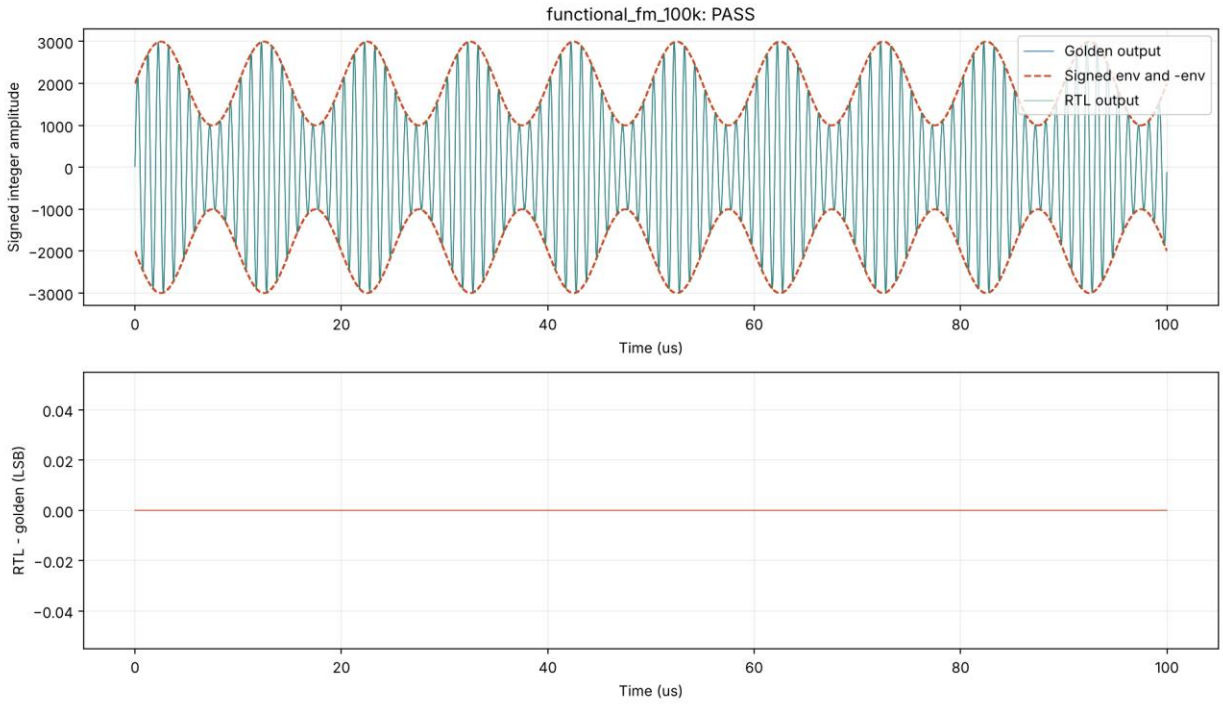


Wideband Message ($f_{msg} = 100$ kHz):

Spectrum:

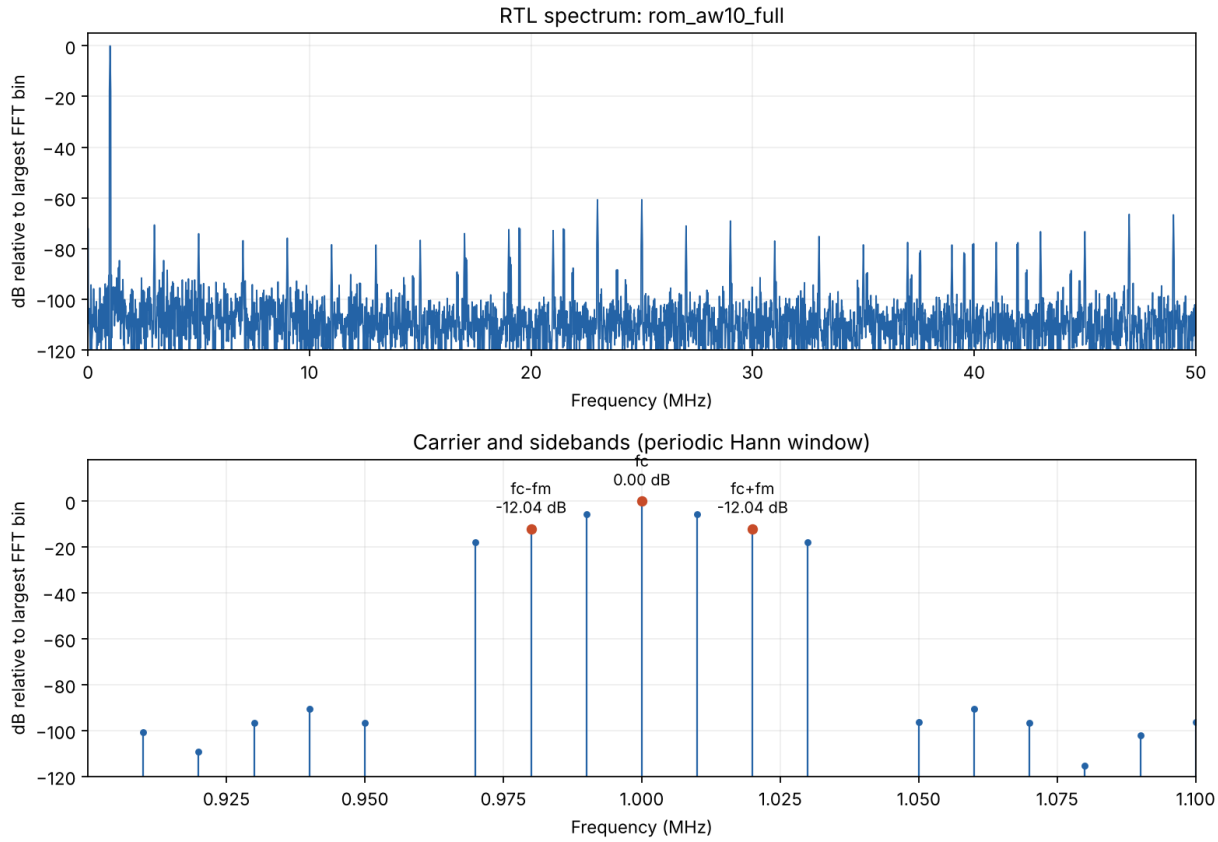


Waveform:

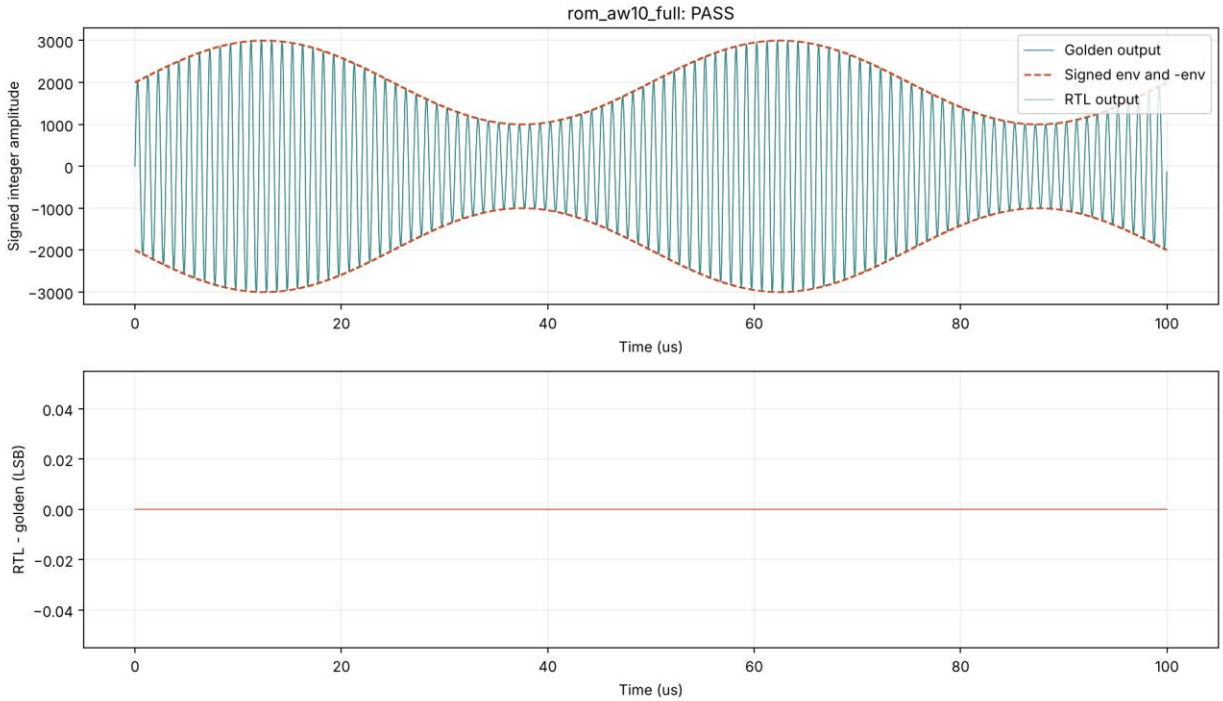


Full-Wave ROM Architecture ($AW = 10$):

Spectrum:

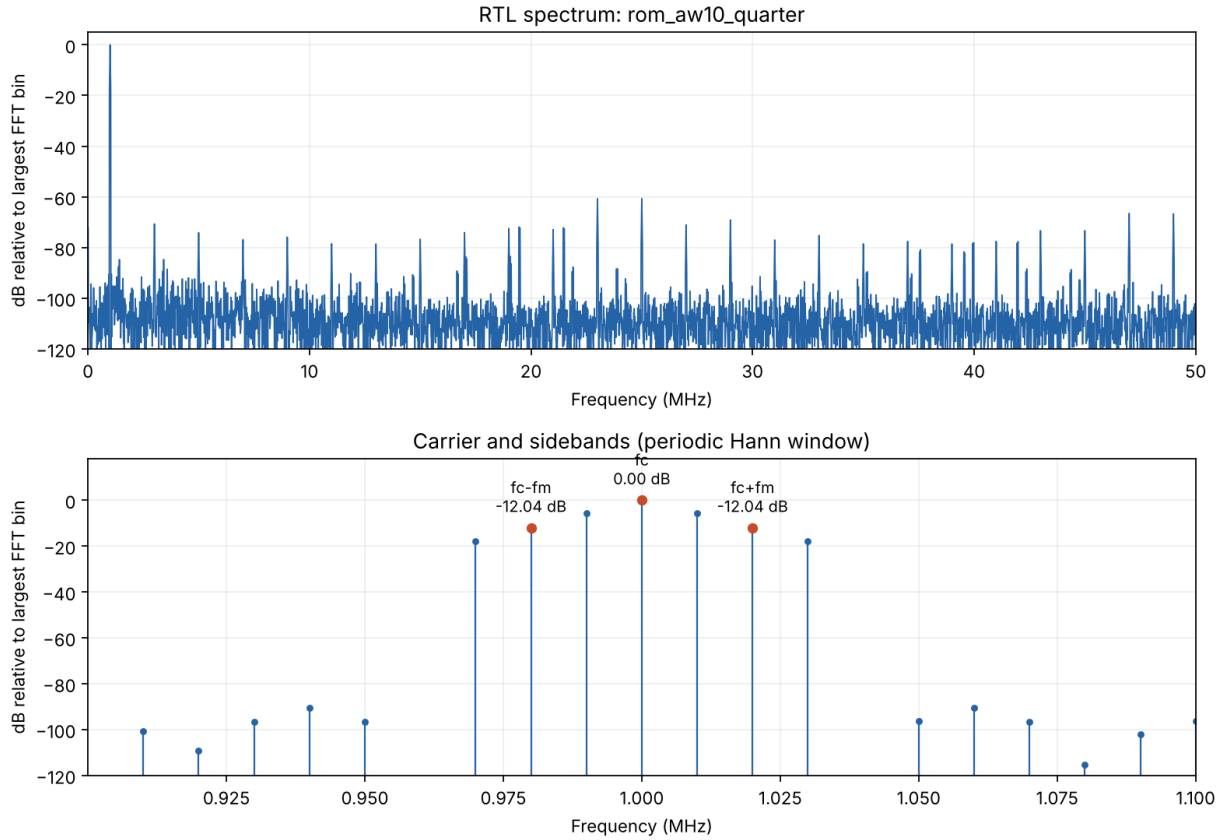


Waveform:

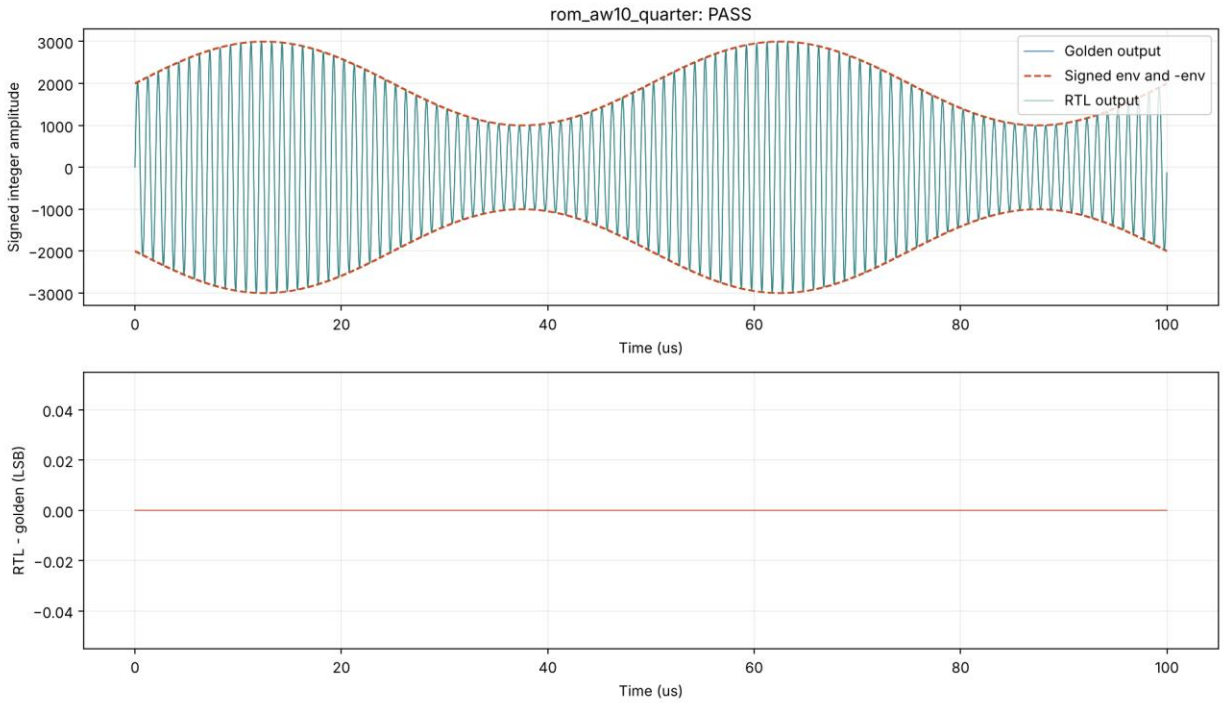


Quarter-Wave ROM Architecture ($AW = 10$):

Spectrum:

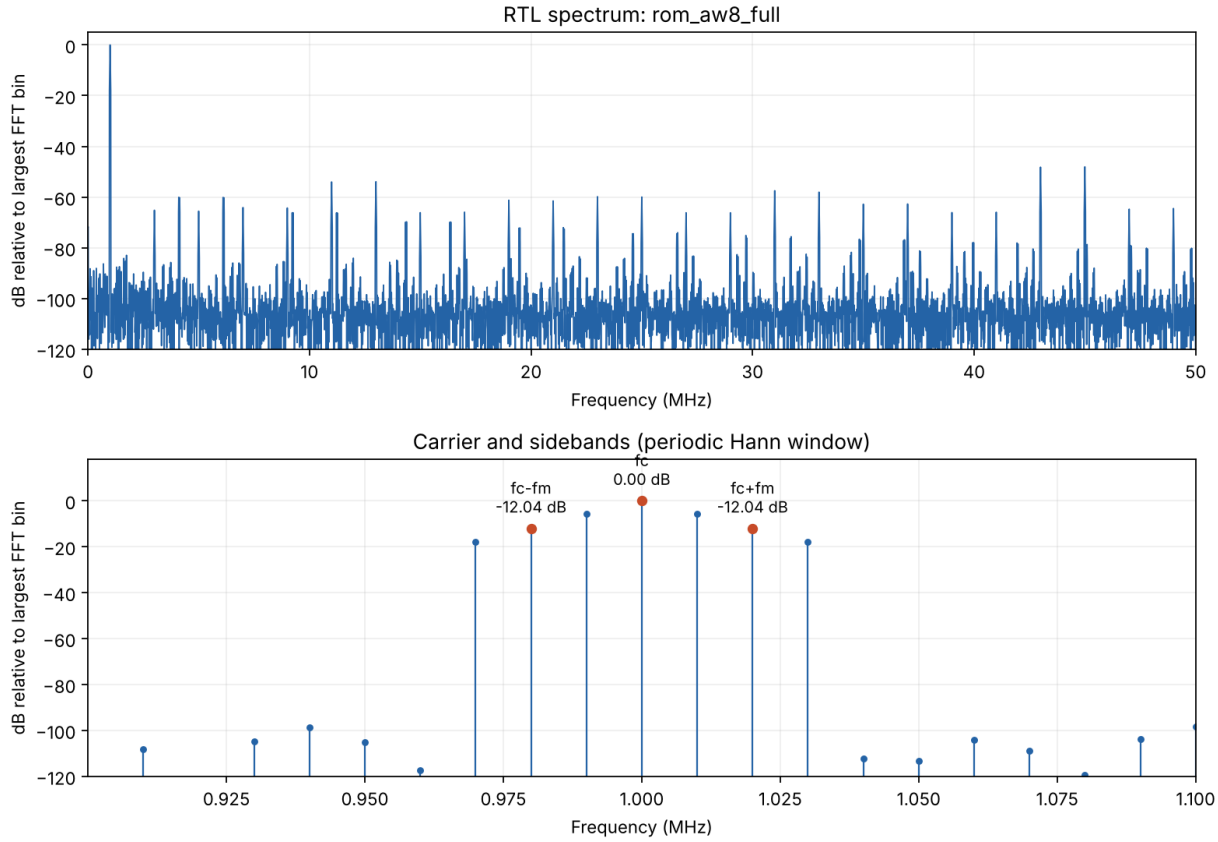


Waveform:

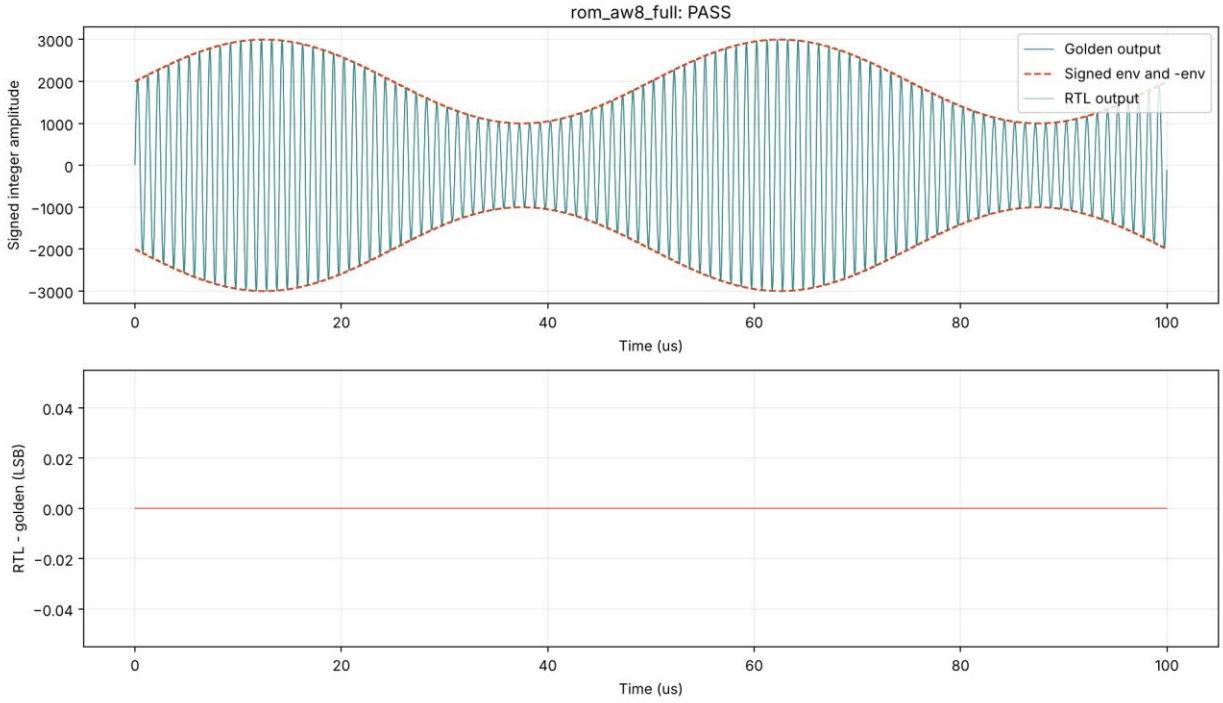


Full-Wave ROM architecture at AW = 8.

Spectrum:

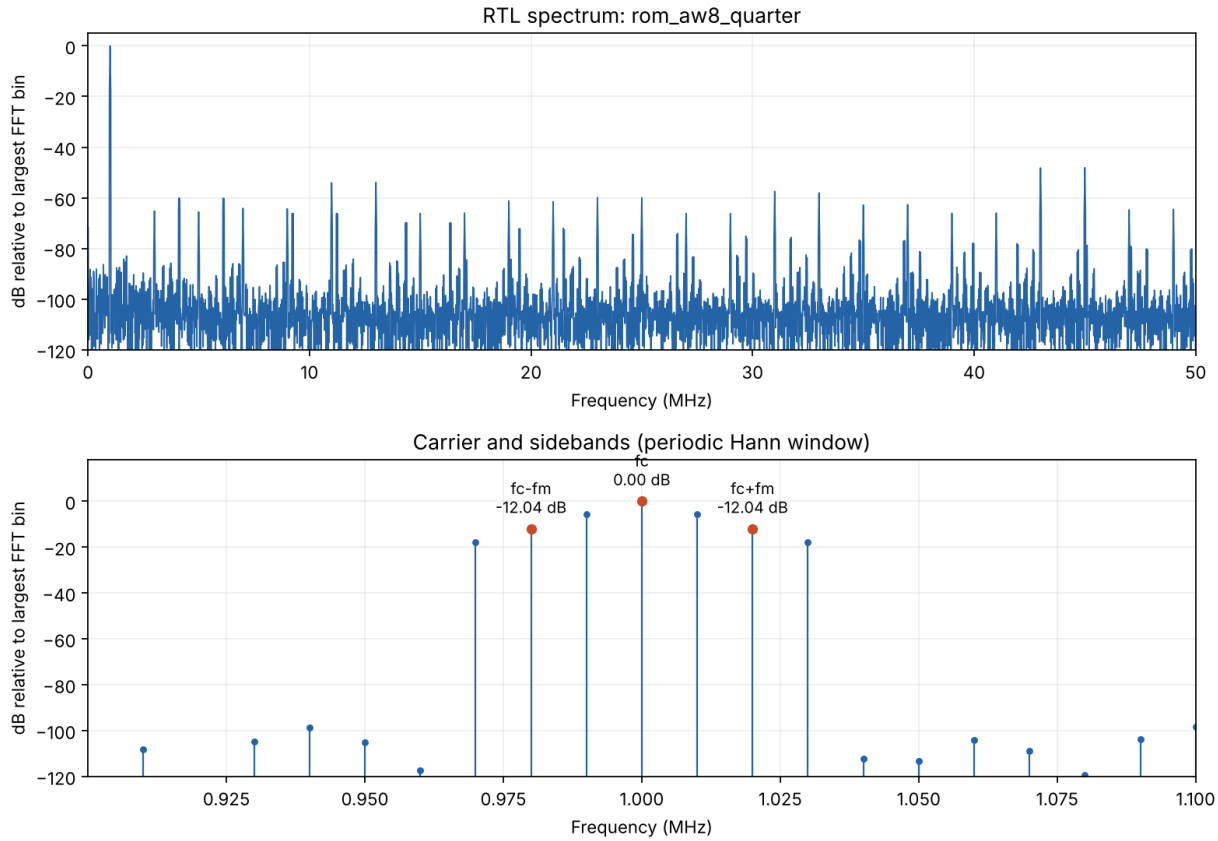


Waveform:

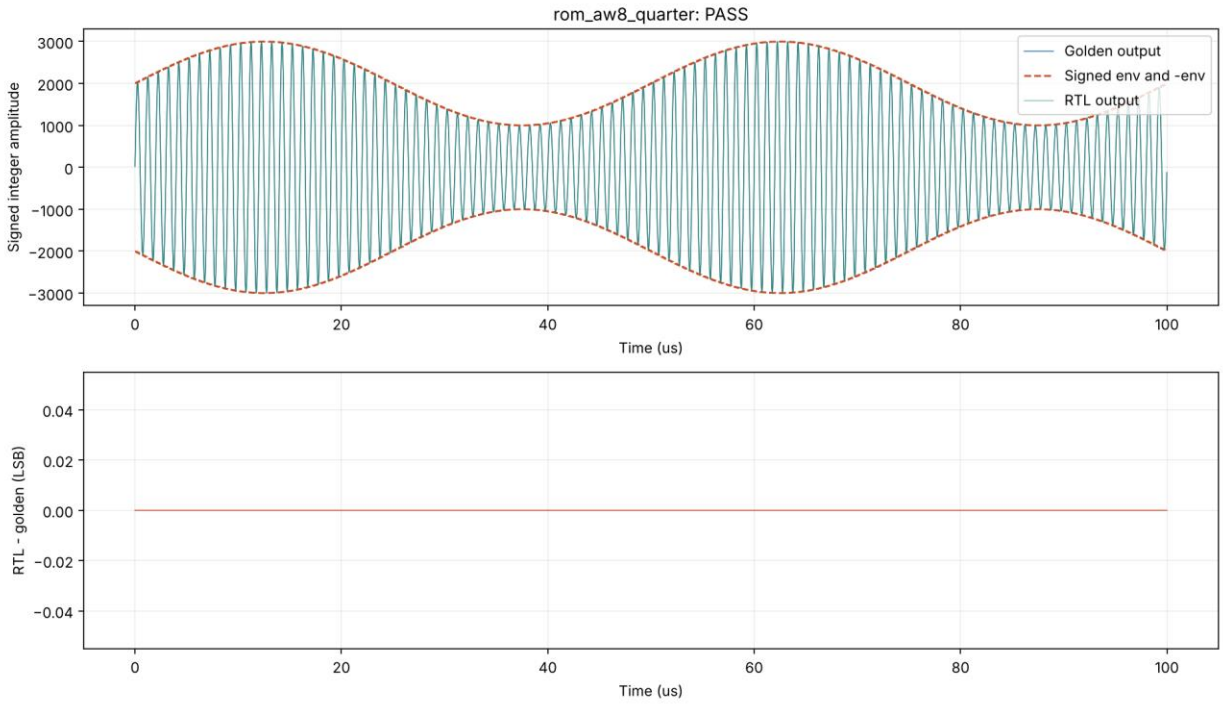


Quarter-Wave ROM architecture at AW = 8

Spectrum:

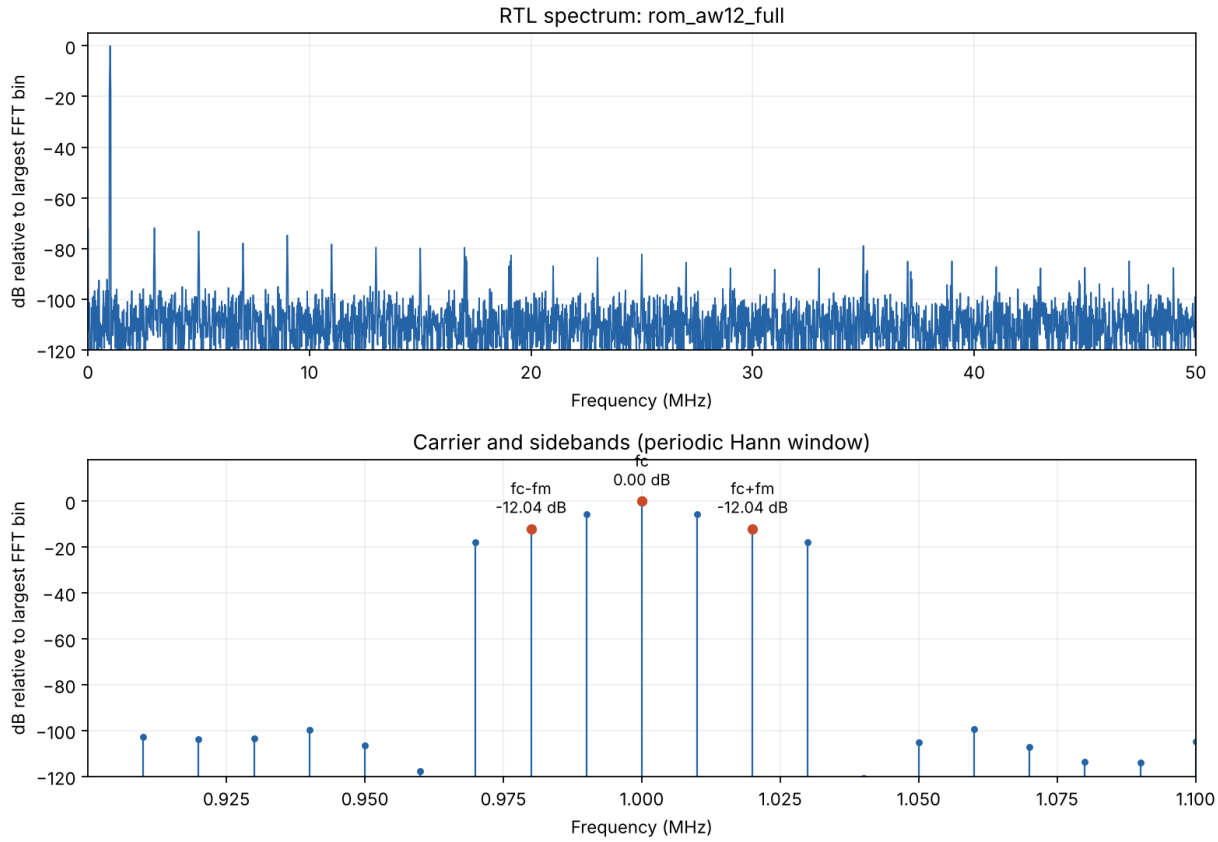


Waveform:

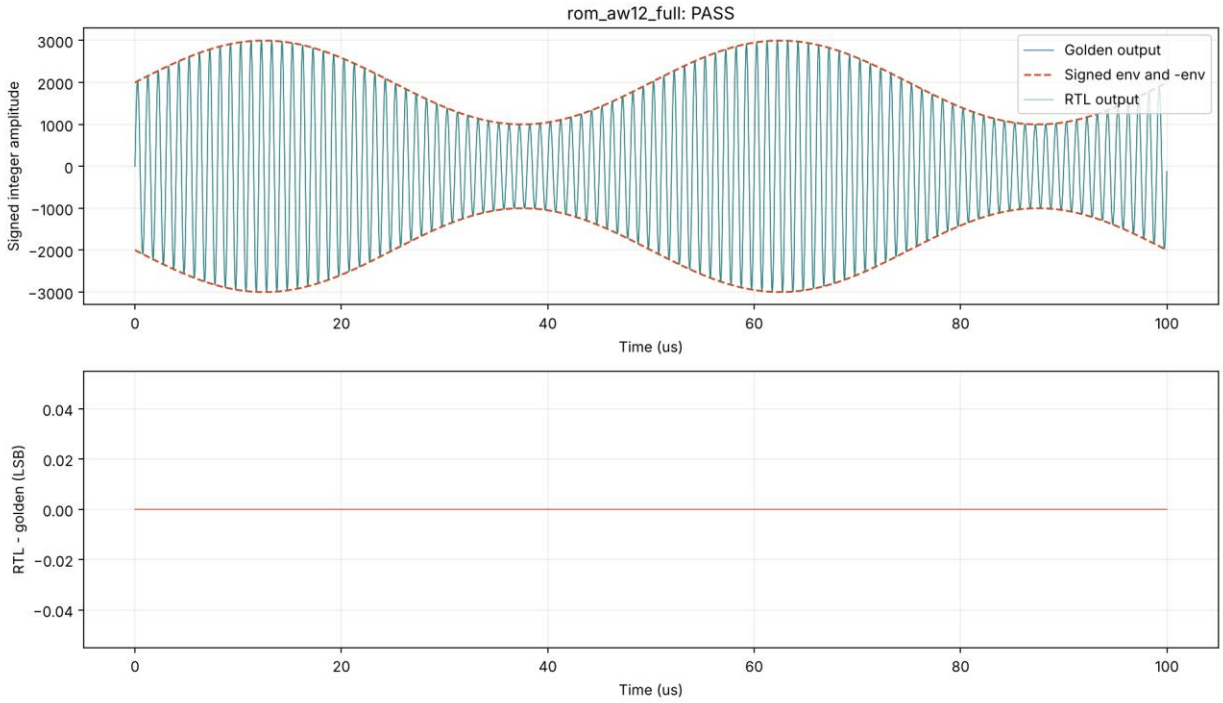


Full-Wave ROM architecture at AW = 12

Spectrum:

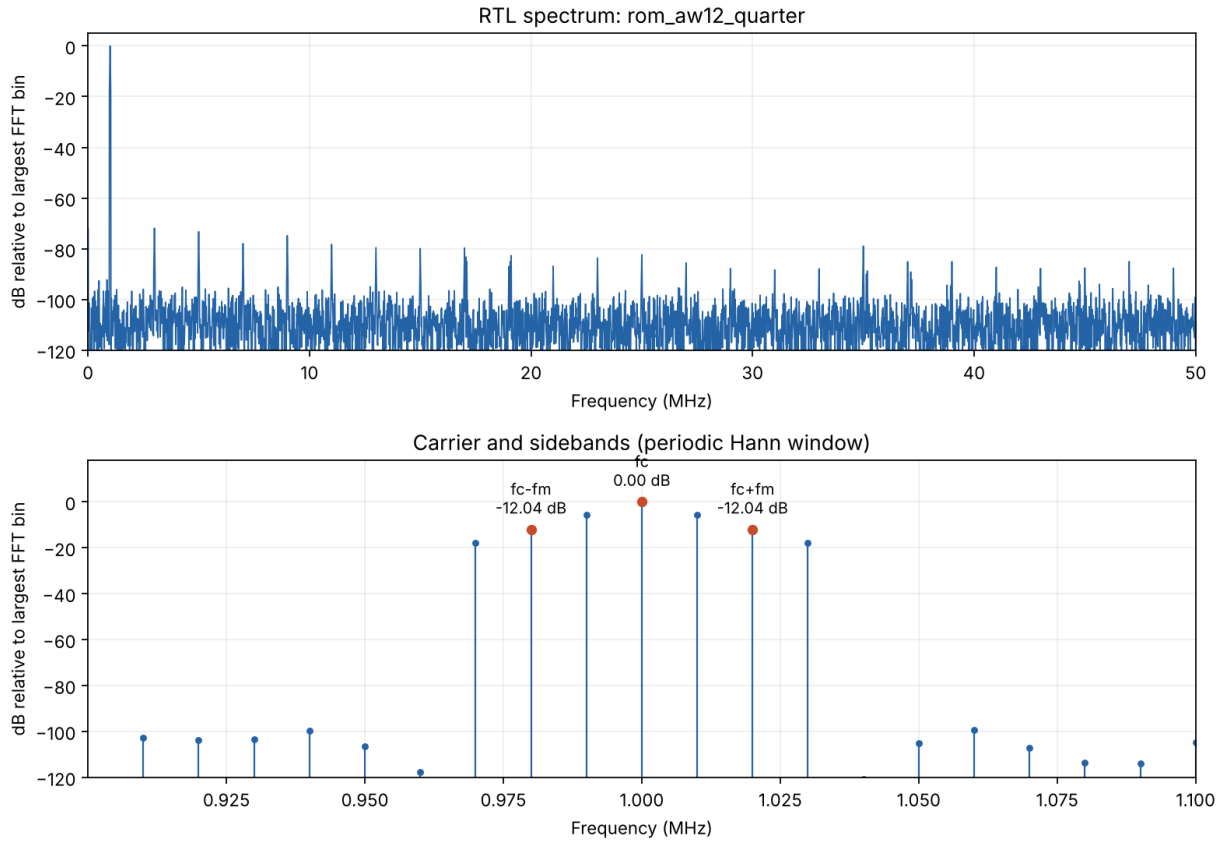


Waveform:

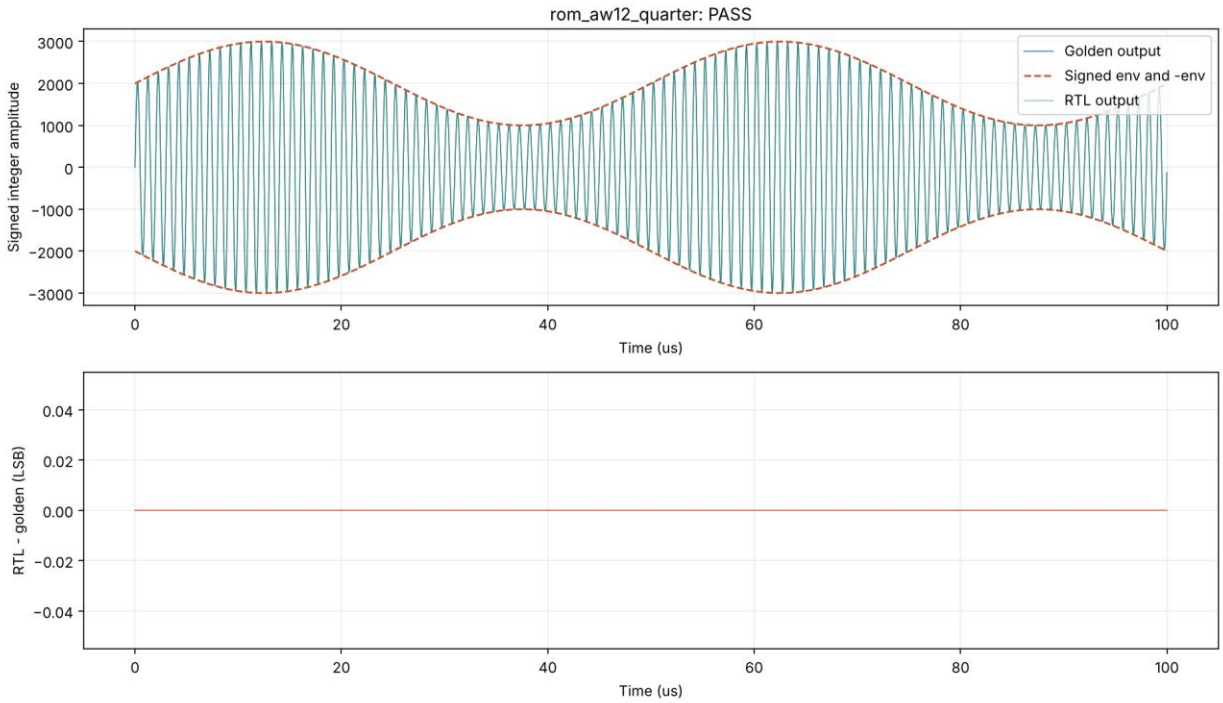


Quarter-Wave ROM architecture at AW = 12

Spectrum:



Waveform:



4.3. Python Unit Tests

Ran test suite of 21 unit tests (python -m unittest discover -s tests).
The script gen_sine_rom.py verifies the midpoint formula to ensure absolute symmetry between Full ROM and Quarter ROM across all memory addresses.

5. SYNTHESIS RESULTS & ANALYSIS

5.1. Hypotheses Under Test

1. **Hypothesis H1 (Signal Accuracy):** Both Full-wave ROM and Quarter-wave ROM architectures must generate completely identical sample data sequences.
   Confirmation: Successfully verified via simulation (Section 4.2) across all configurations AW{8,10,12}.
2. **Hypothesis H2 (Trading Memory Resources for Logic):** Quarter-wave ROM saves approximately 75% of ROM capacity but requires additional address folding logic; thus, total LUT count may not decrease for small ROM table sizes.

3. **Hypothesis H3 (Maximum Operating Frequency Fmax):** Fmax of Quarter-wave ROM may slightly decrease due to the hardware path carrying the additional propagation delay of 2's complement negation logic.

5.2. Vivado Batch Sweep Synthesis Results Frame

Tệp thực thi synth.tcl quét tự động các cấu hình thiết kế tại tần số 100 MHz:

AW	ROM Arch	LUT	FF
8	Full	Pending	Pending
8	Quarter	Pending	Pending
10	Full	Pending	Pending
10	Quarter	Pending	Pending
12	Full	Pending	Pending
12	Quarter	Pending	Pending

6. LIMITATIONS

1. **Testing Scope:** Currently limited to Simulation and RTL Synthesis; not yet executed directly on physical FPGA hardware (Hardware-in-the-loop).
2. **Modulation Architecture:** Only supports DSB-FC transmission. Does not integrate an AM Demodulator/detector, and DSB-SC mode is achieved only by configuring attenuation amp_car = 0.
3. **Test Signal:** Message signal m(t) is currently a single-tone sinusoid generated from a second NCO, not a real-world audio input.
4. **Phase Truncation:** Extracting 2AW memory address bits from a 32-bit accumulator creates Phase Truncation Spurs, limiting spectral purity. The SFDR metric was analyzed via the spectrum.py script.

7. EVALUATION & COMPARISON

7.1. Comparison between Full-Wave ROM and Quarter-Wave ROM Architectures

Based on evaluation of hardware design hypotheses:

- **Memory Resource Efficiency:** Quarter-wave ROM architecture only utilizes 2AW-2 elements, saving precisely 75% of memory storage capacity (ROM footprint) compared to traditional Full-wave ROM.
- **Memory-versus-Logic Trade-off:** Despite substantial memory savings, Quarter-wave ROM requires extra logic circuits for address folding and sign inversion. Consequently, for small table configurations (AW=8), LUT logic count may not decrease significantly as additional logic offsets saved memory.
- **Maximum Operating Frequency (Fmax):** Carrying additional propagation delay for quarter-wave selection logic and sign inversion causes Fmax of Quarter-wave architecture to trend slightly lower than direct Full-wave ROM, though it comfortably satisfies system 100MHz timing constraints.
- **Signal Accuracy (Hypothesis H1):** Both architectures yield bit-exact identical sample data sequences across all verification configurations.

7.2. Comparison between ROM-based NCO Approach and Algorithmic Approximation Approach (CORDIC / Polynomial)

- **ROM-based NCO (Chosen approach in this project):**
  - *Advantages:* Ultra-fast processing speed, low fixed latency (1 sample/clock throughput post-pipeline), simple hardware design, easy bit-exact verification against software models.
  - *Disadvantages:* Constrained by ROM table size and subject to phase truncation spurs.
- **CORDIC / Algorithmic Approximation:**
  - *Advantages:* Does not consume large ROM memory footprints; achieves high precision depending on iteration depth.
  - *Disadvantages:* High logic resource consumption (LUT/FF or DSP blocks) and extended latency (requires multi-cycle clock delays). For standard AM modulation applications, the ROM-based NCO approach is superior in terms of hardware simplicity

TEAM MEMBERS' CONTRIBUTION

Class: IC2010		**Team: 3**		**Contribution**
#	**Student ID**	**Student Name**	**Task**	**%**
1	SE205188	CAO HỮU AN	Wrote sine wave generator block in Verilog including common/phase_acc.v, full_rom/sine_rom_full.v, quarter_rom/sine_rom_quarter.v, nco.v.	16%
2	SE204386	PHẠM MINH TÙNG	Created overall project draft, synthesized and debugged module files, integrated all modules	20%
3	SE205068	MAI THẾ TÂM	Wrote golden reference model in Python for circuit evaluation	16%
4	SE202014	LÊ THẾ TRƯỜNG	Ran synthesis, collected data metrics, authored report	16%
5	SE204511	NGUYỄN HUỲNH MINH TRIẾT	TESTBENCH	16%

6	SE204664	PHAN ANH MINH	Wrote calculation blocks and complete top integration including file sam_envelope.v, am_multiplier.v, full_rom/top_module.v, quarter_rom/top_module.v.	16%
Total:				**100%**

REFERENCES

1. A. V. Oppenheim and R. W. Schafer, *Discrete-Time Signal Processing*, 3rd ed. Upper Saddle River, NJ: Prentice Hall, 2009. (Foundational document on digital signal processing, sampling theory, and modulation).
2. U. Meyer-Baese, *Digital Signal Processing with Field Programmable Gate Arrays*, 4th ed. Berlin, Germany: Springer-Verlag, 2014. (In-depth reference on implementing DSP algorithms, NCOs, and ROM lookups on FPGAs).
3. P. O'Leary, *Fixed-Point Arithmetic: Theory and Practice*, New York, NY: Springer, 2017. (Theoretical foundation on fixed-point math, bit-width management, and bit-shift/overflow error handling in RTL).
4. AMD Xilinx, *LogiCORE IP Direct Digital Synthesizer (DDS) Compiler v6.0 Product Guide*, PG141, 2020. (Xilinx technical documentation on optimizing NCO/DDS architectures for BRAM and LUT efficiency on Xilinx FPGAs).
5. *IEEE Standard for Verilog Hardware Description Language*, IEEE Std 1364-2005, 2006. (Industry standard for Verilog HDL language used throughout this project).
6. L. Cordesses, "Direct digital synthesis: A tool for periodic wave generation (part 1 & 2)," *IEEE Signal Processing Magazine*, vol. 21, no. 4, pp. 50–54, Jul. 2004. (Paper analyzing sine wave generation via phase accumulators and ROM sizing optimization).
7. J. Tierney, C. Rader, and B. Gold, "A digital frequency synthesizer," *IEEE Transactions on Audio and Electroacoustics*, vol. 19, no. 1, pp. 48–57, Mar. 1971. (Seminal paper establishing the foundations of NCO/DDS architectures).
8. S. Palnitkar, *Verilog HDL: A Guide to Digital Design and Synthesis*, 2nd ed. Upper Saddle River, NJ: Prentice Hall, 2003. (Standard guide for writing synthesizable Verilog RTL and building testbenches).
9. C. Spear and G. Tumbush, *SystemVerilog for Verification: A Guide to Learning the Testbench Language Features*, 3rd ed. New York, NY: Springer, 2012. (Reference guide on hardware verification methodology and building self-checking testbenches).
10. AMD Xilinx, *Vivado Design Suite User Guide: Synthesis*, UG901, 2023. (Guide for hardware synthesis, LUT/FF/BRAM resource optimization, and timing analysis on Artix-7 FPGAs).